Name :- Yug Kapuriya

UNID :- UMID13012677975

Gmail :- yugkapuriya404@gmail.com

linkedIn :- Yug Kapuriya

Custom Payload Encoder & Obfuscation Framework

Technical Documentation

A practical educational toolkit for payload encoding and evasion testing

Ethical Use Only | Authorized Environments Only

Modules Included

1.Encoder (Base64 / XOR / ROT13 / Multi-Layer)

2. Obfuscator (hex/Reverse/Toggle/Insert)

3. Evasion Tester (signature checks)

4. Report Generator (payload_report.txt)

PROJECT OVERVIEW

Purpose

- Study how offensive payloads are transformed to evade detection
- Understand encoding techniques: Base64, XOR, ROT13 and Multi-Layer
- Simulate evasion testing against basic static signature-based detection
- Compare original vs. encoded/obfuscated payloads side by side
- Generate structured audit reports with evasion insights and recommendations

Practical Motivation

Adversaries frequently modify their payloads to avoid detection by security tools such as Antivirus, EDR, IPS, and Firewalls. Understanding these methods is crucial for:

- Red team offensive workflows and penetration testing
- Blue team defensive improvements and detection rule tuning
- Malware analysis education in controlled lab environments
- Signature evasion research and security awareness

Tools and Technologies

Technology	Purpose
Python 3.x	Core programming language
base64	Base64 encoding and decoding module
itertools / random	Character manipulation and obfuscation
hashlib	Hashing functions (used in evasion testing)
argparse	Optional CLI argument parsing

SYSTEM ARCHITECTURE

Module Structure

File	Module	Function
main.py	Automation	Runs all modules in sequence
encoder.py	Encoder	Base64 / XOR / ROT13 / Multi-layer
obfuscator.py	Obfuscator	Hex / Reverse / Toggle / Insert
evasion_tester.py	Evasion Tester	signature detection simulation
report_generator.py	Report Generator	Saves payload_report.txt

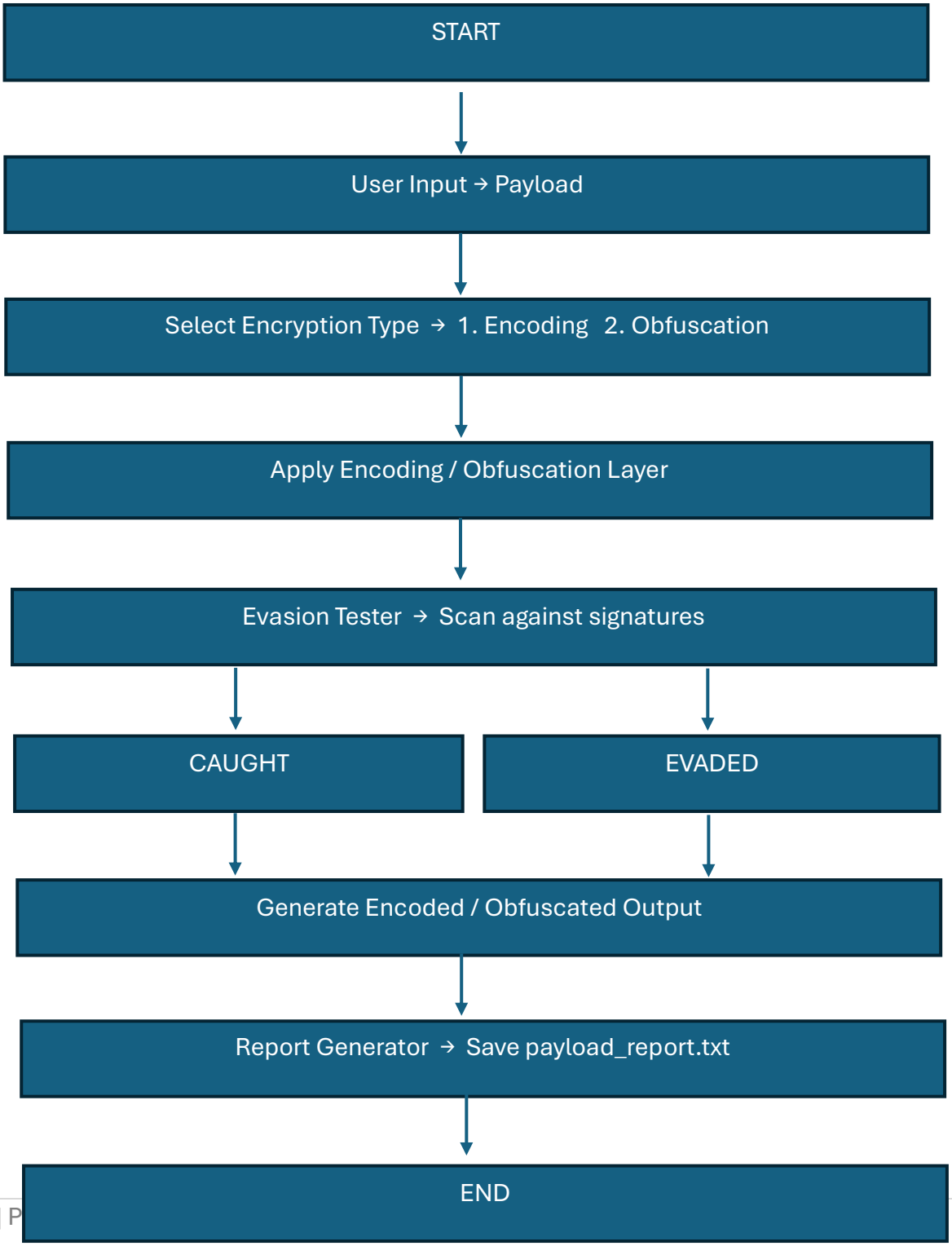
Project Objectives

- Build a payload encoder supporting Base64, XOR, and ROT13 encoding
- Implement multiple string obfuscation techniques
- Simulate evasion testing using pattern-matching detection logic
- Compare original vs. obfuscated payloads and highlight effectiveness
- Generate a final report with evasion success rates and recommendations

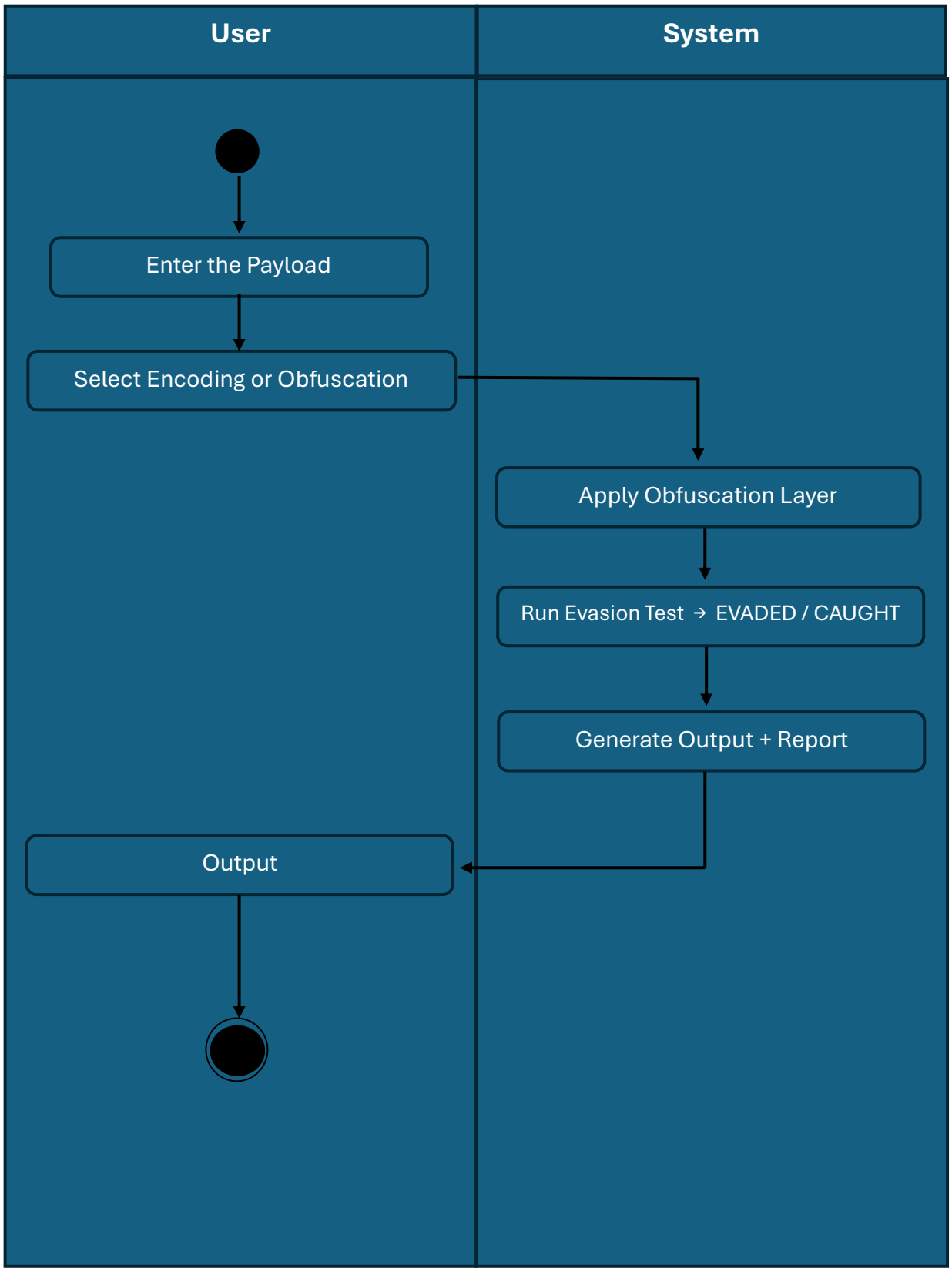
Diagrams

System Flow Diagram

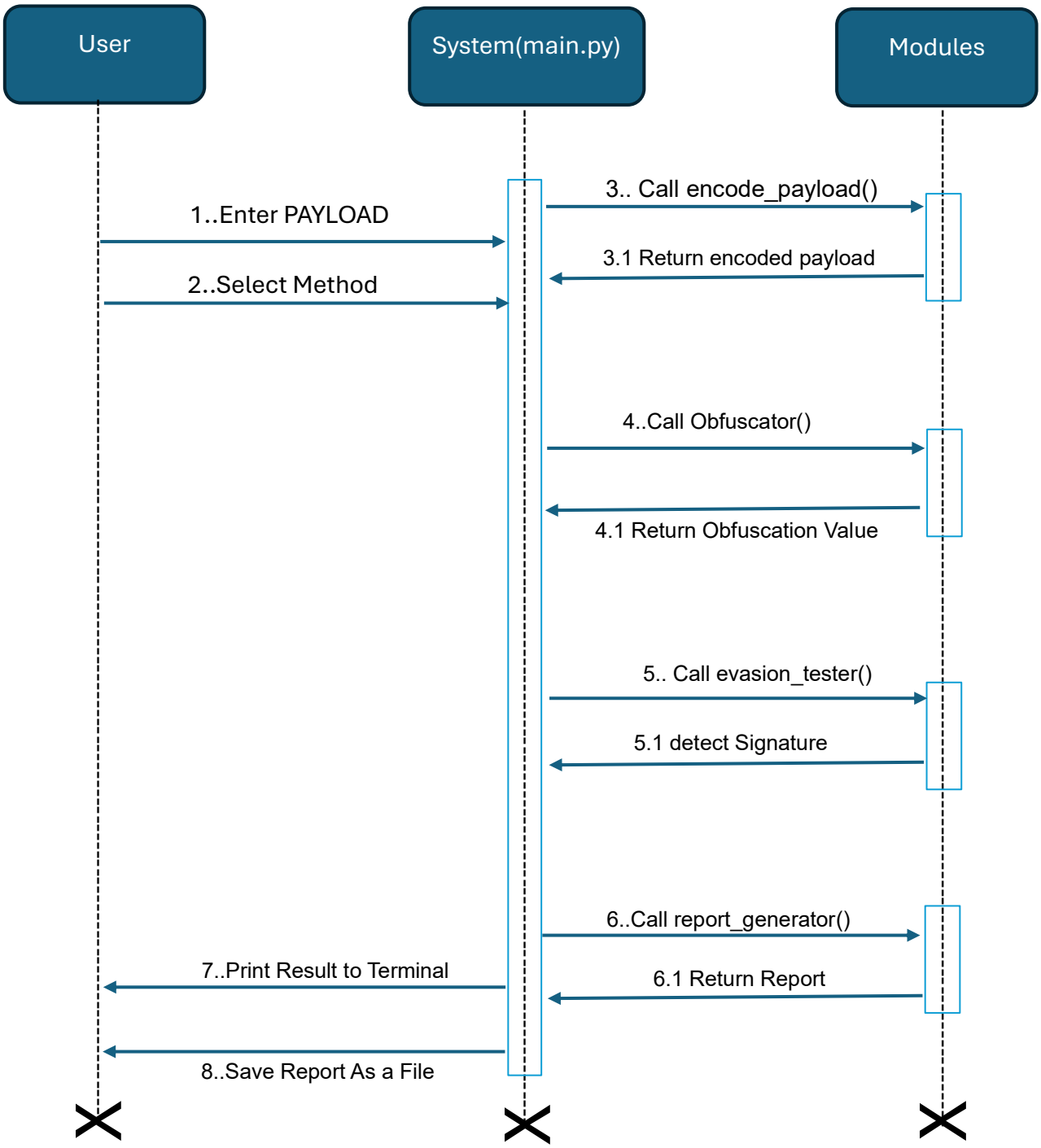
The flowchart below shows the complete execution flow from user input to final report.



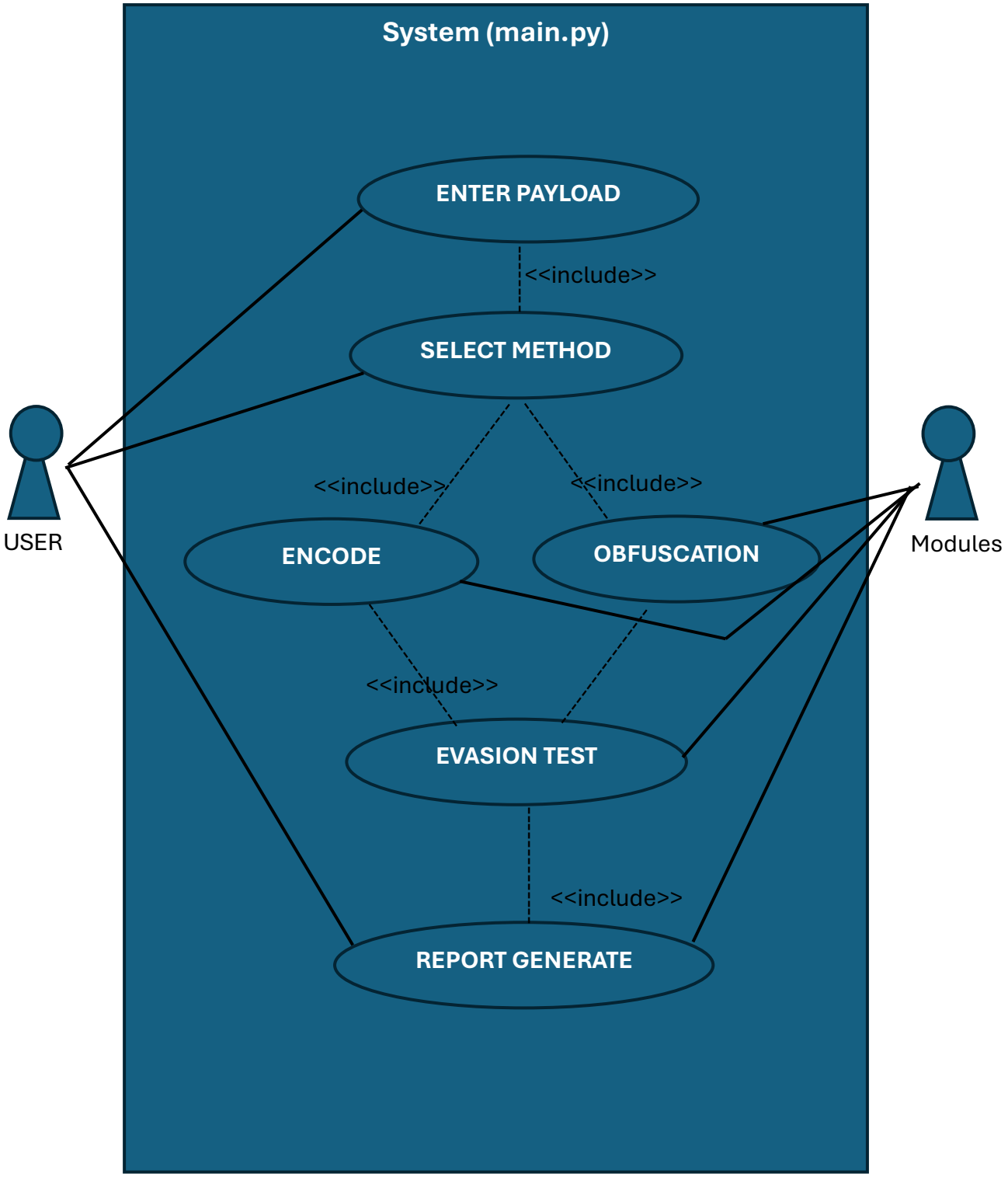
Activity Diagram



Sequence Diagram



Use Case Diagram



MODULE DESCRIPTIONS

Module 1 — Encoder

Encodes the payload using the selected method. Supports Base64, XOR (user-defined key), ROT13, and Multi-Layer (Base64 + XOR + ROT13 combined).

Item	Detail
File	encoder.py
Function	encode_payload(payload)
Methods	Base64, XOR (key), ROT13, Multi-Layer
Input	Raw payload string + method choice (menu)
Output	Encoded string + method name tuple
Multi-Layer Order	Base64 → XOR → ROT13
XOR Key	Asked only when XOR is selected

Module 2 — Obfuscator

Applies string obfuscation techniques to make payloads unrecognizable to simple pattern matching and signature scanning.

Item	Detail
File	obfuscator.py
Function	obfuscate_payload(payload, method)
Hex Escape	Converts every character to \xNN hex code
Reverse	Flips the entire string backwards
Case Toggle	Alternates UPPER and lower characters

Module 3 — Evasion Tester

Simulates signature-based detection. Scans both original and encoded payloads against a database of known malicious keywords and compares detection results.

Item	Detail
File	evasion_tester.py
Function	run_evasion_test(original, encoded)
Signatures	keywords: powershell, whoami, cmd.exe, wget, curl, eval, exec, shellcode...
Scan Original	Checks if original payload matches any signature
Scan Encoded	Checks if encoded payload matches any signature
Result	EVADED (bypass successful) or CAUGHT (detected)

Module 4 — Report Generator

Compiles all encoding and evasion results into a formatted audit report and saves it as payload_report.txt in the project folder.

Item	Detail
File	report_generator.py
Function	generate_report(results)
Payload	Original payload tested
Method	Encoding or obfuscation method used
Evasion Result	EVADED or CAUGHT per test
Summary	Total tested, evaded count, evasion rate %
Recommendations	4 actionable security improvement tips

PRACTICAL TECHNIQUES

Offensive Techniques

- Payload mutation and transformation using encoding layers
- Basic encryption (XOR) and substitution ciphers (ROT13)
- Breaking signature-based detection through obfuscation
- Pattern evasion testing against simulated antivirus signatures
- Multi-layer encoding for maximum signature bypass effectiveness

Defensive Techniques

- Understanding limitations of static signature-only detection
- Demonstrating why layered security (signature + behavioral) is necessary
- Recognizing common obfuscation patterns used by real attackers
- Strengthening detection rules based on evasion test results

Encoding vs Obfuscation Comparison

Technique	Method	Evasion Effectiveness
Base64	Encodes to alphanumeric string	High — removes all keywords
XOR	Scrambles with numeric key	High — produces non-printable chars
ROT13	Shifts letters by 13 positions	Medium — only shifts alphabet
Multi-Layer	Base64 + XOR + ROT13 combined	Very High — multiple transforms
Hex Escape	Converts to \xNN hex codes	High — hides readable text
Reverse	Flips string backwards	Low — keywords still detectable
Case Toggle	Alternates upper/lower case	Low — keywords still present

SECURITY NOTES & RECOMMENDATIONS

Security Recommendations

- **Signature detection alone is easily bypassed — never use as sole defense**
- Combine behavioral analysis + signature matching for layered security
- Update signature databases regularly with new payload samples
- Implement allow-listing in addition to deny-listing for critical systems
- Monitor network traffic for unusual patterns even without keyword matches

Ethical & Legal Disclaimer

IMPORTANT — Authorized Use Only

- Authorized penetration testing with written permission only
- Educational cybersecurity lab environments
- Security auditing of systems you own

Unauthorized use is illegal under CFAA, UK Computer Misuse Act, and equivalent laws.

HOW TO RUN

Step 1 — Place all 5 files in the same folder

- main.py
- encoder.py
- obfuscator.py
- evasion_tester.py
- report_generator.py

Step 2 — Open terminal and run

python main.py

Step 3 — Follow the prompts

- Enter the payload you want to encode (e.g. powershell -enc whoami)
- Select encryption type: 1 for Encoding, 2 for Obfuscation
- Select specific method from the sub-menu that appears
- If XOR is selected, enter your numeric key when prompted

Step 4 — View the output

- Results print to the terminal step by step
- Evasion result shows EVADED or CAUGHT with signature details
- Final report saved as payload_report.txt in the same folder